

EX.NO.1	STUDY OF HTML TAGS
DATE: 29.06.26	

AIM:

To study about different HTML tags and their usage in webpage creation.

DESCRIPTION:

HTML (HyperText Markup Language) is the standard markup language used to create web pages. It defines the structure of a webpage using different tags. Browsers interpret these tags and display the webpage accordingly.

HTML consists of:

- **Elements (tags)**
- **Attributes**
- **Content**

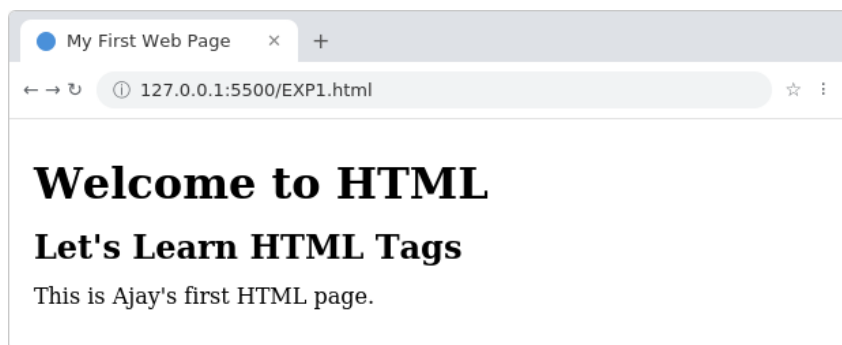
HTML tags are predefined keywords enclosed within angle brackets (< >) that tell the browser how to display or organize the content on a web page.

Most HTML tags have:

- **Opening tag**
- **Content**
- **Closing tag**

HTML SYNTAX:

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Web Page</title>
</head>
<body>
  <h1>Welcome to HTML</h1>
  <h2>Let's Learn HTML Tags</h2>
  <p>This is Ajay's first HTML page.</p>
</body>
</html>
```

Output:**TAGS in HTML:**

1. Heading Tag (<h1> to <h6>) : Displays headings with different font sizes.

Syntax:

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>
```

Output:

Heading 1

Heading 2

Heading 3

Heading 4

Heading 5

Heading 6

2. Paragraph Tag (<p>) : Displays text as a paragraph.**Syntax:**

```
<p>
HTML (HyperText Markup Language) is the standard language used to
create web pages. It provides the basic structure of a webpage and
works together with CSS and JavaScript to build attractive and
interactive websites.
</p>
```

Output:

HTML (HyperText Markup Language) is the standard language used to create web pages. It provides the basic structure of a webpage and works together with CSS and JavaScript to build attractive and interactive websites.

**3. Line Break Tag (
) :** Inserts a line break.**Syntax:**

```
HTML<br>
CSS<br>
JavaScript
```

Output:

HTML
CSS
JavaScript

4. Horizontal Rule Tag (<hr>) : Inserts a horizontal line.**Syntax:**

```
<p>First Section</p>
<hr>
<p>Second Section</p>
```

Output:

First Section

Second Section

5. Bold Tag () : Displays bold text.**Syntax:**`<b>Bold Text</b>`**Output:****Bold Text****6. Italic Tag (<i>) :** Displays italic text.**Syntax:**`<i>Italic Text</i>`**Output:***Italic Text***7. Underline Tag (<u>) :** Underlines text.**Syntax:**`<u>Underlined Text</u>`**Output:**Underlined Text**8. Superscript Tag (<sup>) :** Displays superscript text.**Syntax:**`x<sup>2</sup>`**Output:** x^2 **9. Subscript Tag (<sub>) :** Displays subscript text.**Syntax:**`H<sub>2</sub>O`**Output:** H_2O **10. Anchor Tag (<a>) :** Creates hyperlinks.**Syntax:**`<a href="https://www.google.com">Visit Google</a>`**Output:**[Visit Google](https://www.google.com)**11. Image Tag () :** Displays an image.**Syntax:**``

Output:**12. Ordered List () :** Displays a numbered list.**Syntax:**

```
<ol>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ol>
```

Output:

```
1. HTML
2. CSS
3. JavaScript
```

13. Unordered List () : Displays a bulleted list.**Syntax:**

```
<ul>
  <li>Apple</li>
  <li>Mango</li>
  <li>Orange</li>
</ul>
```

Output:

```
• Apple
• Mango
• Orange
```

14. Table Tag (<table>) : Creates tables.**Syntax:**

```
<table border="1">
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Rahul</td>
    <td>20</td>
  </tr>
</table>
```

Output:

Name	Age
Rahul	20

15. Form Tag (<form>) : Creates user input forms.**Syntax:**

```
<form>
```

Name:

```
<input type="text"><br><br>
<input type="submit" value="Submit">
</form>
```

Output:



16. Input Tag (<input>) : Creates an input field.

Syntax:

```
<input type="text" placeholder="Enter Name">
```

Output:

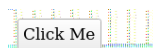


17. Button Tag (<button>) : Creates a clickable button.

Syntax:

```
<button>Click Me</button>
```

Output:



18. Division Tag (<div>) : Groups HTML elements into sections.

Syntax:

```
<div>
  <h2>Student Details</h2>
  <p>Name: Ajay</p>
</div>
```

Output:

Student Details

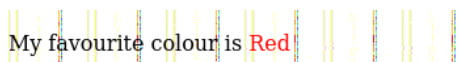
Name: Ajay

19. Span Tag () : Groups inline text for styling.

Syntax:

```
<p>
  My favourite colour is
  <span style="color:red;">Red</span>
</p>
```

Output:



20. Marquee Tag (<marquee>) : Displays scrolling text.

Syntax:

```
<marquee>
  Welcome to HTML Laboratory
</marquee>
```

Output:

Welcome to HTML Laboratory

RESULT:

Thus, the basic structure of an HTML document and the usage of various HTML tags such as headings, paragraphs, formatting tags, links, images, lists, tables, forms, and other commonly used HTML elements were studied and executed successfully.

EX.NO.2	SIMPLE BLOG WEBPAGE CREATION USING HTML
DATE: 06.07.26	

AIM:

To design and develop a Personal Digital Profile webpage using HTML and CSS that displays personal information, educational qualifications, technical skills, daily routine, favourite website, favourite quote and contact details..

ALGORITHM:

1. Create an HTML document.
2. Add CSS inside the <style> tag.
3. Create sections for About Me, Skills, Education, Routine, Favourite Website and Contact.
4. Insert profile image.
5. Apply CSS for colours and layout.
6. Save the file as **index.html** and execute it in a browser.

HTML Code:

```
<!DOCTYPE html>
<html>
<head>
<title>My Digital Profile</title>
<style>
body{
font-family:Arial;
background:#f3f3f3;
margin:20px;
}
.container{
width:850px;
margin:auto;
background:white;
padding:20px;
border-radius:10px;
}
h1{
text-align:center;
color:navy;
}
img{
width:180px;
height:200px;
border-radius:10px;
border:2px solid gray;
}
table{
width:100%;
}
.section{
```

```
color:crimson;
}
ul,ol{
font-size:18px;
}
.routine td,.routine th{
border:1px solid black;
padding:8px;
text-align:center;
}
.routine{
border-collapse:collapse;
margin-top:10px;
}
.footer{
text-align:center;
margin-top:20px;
font-size:14px;
}
</style>
</head>
<body>
<div class="container">
<h1>WELCOME TO MY DIGITAL PROFILE</h1>
<table>
<tr>
<td width="30%">

</td>
<td>
<h2 class="section">ABOUT ME</h2>
<p>
Hello! I am <b>Ajay</b>, a Third Year Computer Science and Engineering student at
<b>Sri Ramakrishna Engineering College, Coimbatore.</b>
I enjoy learning Full Stack Development, Artificial Intelligence,
Machine Learning and Data Science.
</p>
</td>
</tr>
</table>
<hr>
<table>
<tr>
<td>
<h2 class="section">MY SKILLS</h2>
<ul>
<li>HTML</li>
<li>CSS</li>
<li>JavaScript</li>
<li>Python</li>
<li>C Programming</li>
```

```

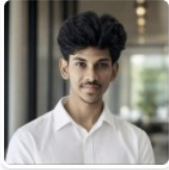
</ul>
</td>
<td>
<h2 class="section">EDUCATION</h2>
<ol>
<li>SSLC</li>
<li>HSC</li>
<li>B.E Computer Science and Engineering</li>
</ol>
</td>
</tr>
</table>
<hr>
<h2 class="section">MY DAILY ROUTINE</h2>
<table class="routine">
<tr>
<th>Time</th>
<th>Activity</th>
</tr>
<tr>
<td>8.00 AM</td>
<td>College</td>
</tr>
<tr>
<td>5.00 PM</td>
<td>Coding Practice</td>
</tr>
<tr>
<td>8.00 PM</td>
<td>Study</td>
</tr>
</table>
<hr>
<h2 class="section">FAVOURITE WEBSITE</h2>
<a href="https://openai.com">OpenAI</a>
<h2 class="section">FAVOURITE QUOTE</h2>
<p><i>"Learning never exhausts the mind."</i></p>
<p><b>— Leonardo da Vinci</b></p>
<hr>
<h2 class="section">CONTACT ME</h2>
<p><b>Name :</b> Ajay</p>
<p><b>College :</b> Sri Ramakrishna Engineering College</p>
<p><b>Department :</b> Computer Science and Engineering</p>
<p><b>City :</b> Coimbatore, Tamil Nadu</p>
<div class="footer">
© 2026 My Digital Profile
</div>
</div>
</body>
</html>

```

OUTPUT:

WELCOME TO MY DIGITAL PROFILE

Roll No: 71812401007



About Me

Hello! I am **Ajay**, a passionate student. I enjoy learning about *Full Stack Development, Artificial Intelligence, Machine Learning* and *Data Science*. I love solving programming challenges and continuously improving my technical skills.

★ MY SKILLS

- HTML
- CSS
- Java
- Python
- C Programming
- JavaScript

🎓 EDUCATIONAL QUALIFICATION

- SSLC
- HSC
- B.E. Computer Science Engineering

🕒 MY DAILY ROUTINE

Time	Activity
8:00 AM	College
2:00 PM	Laboratory
5:00 PM	Self Learning / Coding Practice
8:00 PM	Study

🌐 MY FAVOURITE WEBSITE

Visit : [OpenAI](#)

💬 MY FAVOURITE QUOTE

"Learning never exhausts the mind."

— Leonardo da Vinci

✉ CONTACT ME

Name : Ajay

Email : ajay@example.com

Phone : +91 XXXXXXXXXX

City : Madurai, Tamil Nadu, India

© 2026 My Digital Profile. All Rights Reserved.

RESULT:

Thus, a Personal Digital Profile Web Page was successfully created using HTML and CSS and executed successfully in the web browser.

EX.NO.3	HOTSPOT CREATION
DATE: 13.07.26	

AIM:

To design a webpage that displays an image of the India map with clickable hotspots using HTML Image Maps. Clicking on different regions should navigate to their respective Wikipedia pages.

ALGORITHM:

1. Create an HTML file.
2. Create a CSS file for styling.
3. Add the India map image using the tag.
4. Associate the image with an image map using the usemap attribute.
5. Create a <map> element.
6. Define hotspot areas using the <area> tag.
7. Specify the coordinates of each hotspot.
8. Provide Wikipedia links for each state.
9. Apply CSS styling.
10. Save the files.
11. Execute the HTML file in a web browser.
12. Verify that clicking each hotspot opens the correct webpage.

HTML Code:

```
<!DOCTYPE html>
<html>
<head>
<title>Ajay's India Map Hotspots</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<h1>INDIA MAP HOTSPOTS</h1>
<h2>Ajay's Statewise Wikipedia Finder</h2>
<p>
Click on any highlighted state to know more about it.
</p>

<map name="india">
<!-- Jammu & Kashmir -->
<area
shape="circle"
coords="378,106,22"
href="https://en.wikipedia.org/wiki/Jammu_and_Kashmir"
target="_blank"
alt="Jammu and Kashmir">

<!-- Delhi -->
<area
shape="circle"
coords="429,323,15"
```

```
href="https://en.wikipedia.org/wiki/Delhi"
target="_blank"
alt="Delhi">

<!-- Odisha -->
<area
shape="circle"
coords="520,470,18"
href="https://en.wikipedia.org/wiki/Odisha"
target="_blank"
alt="Odisha">

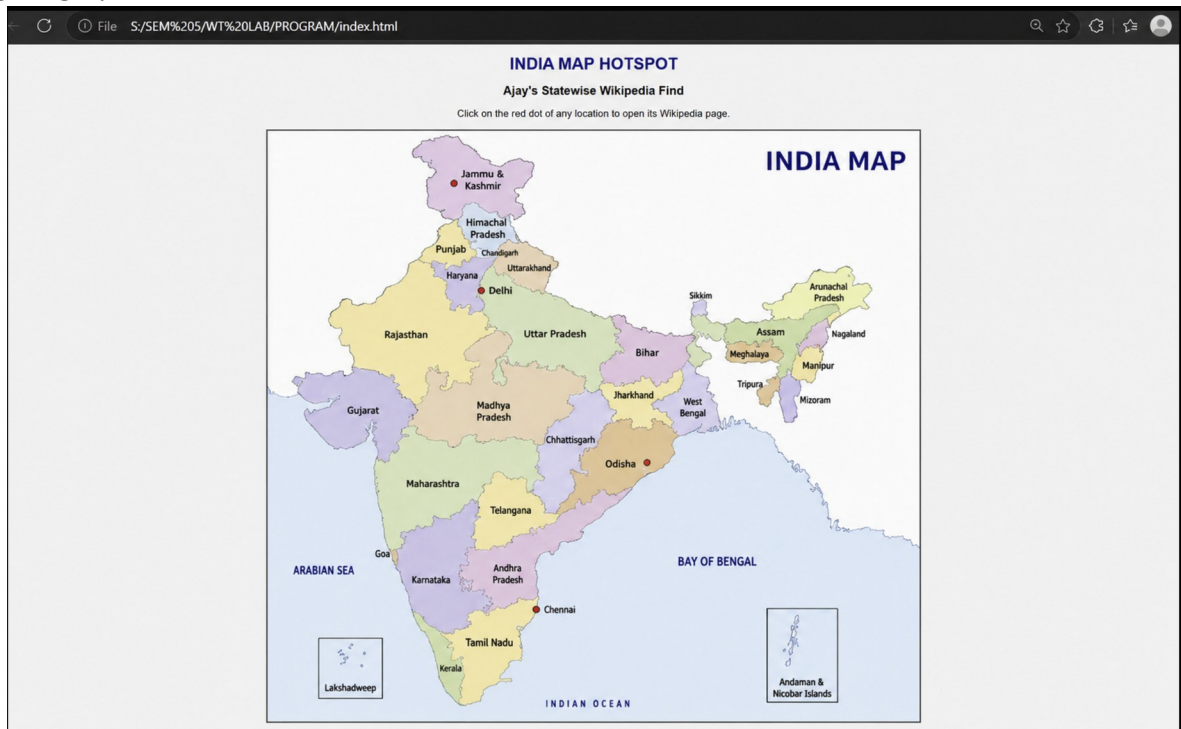
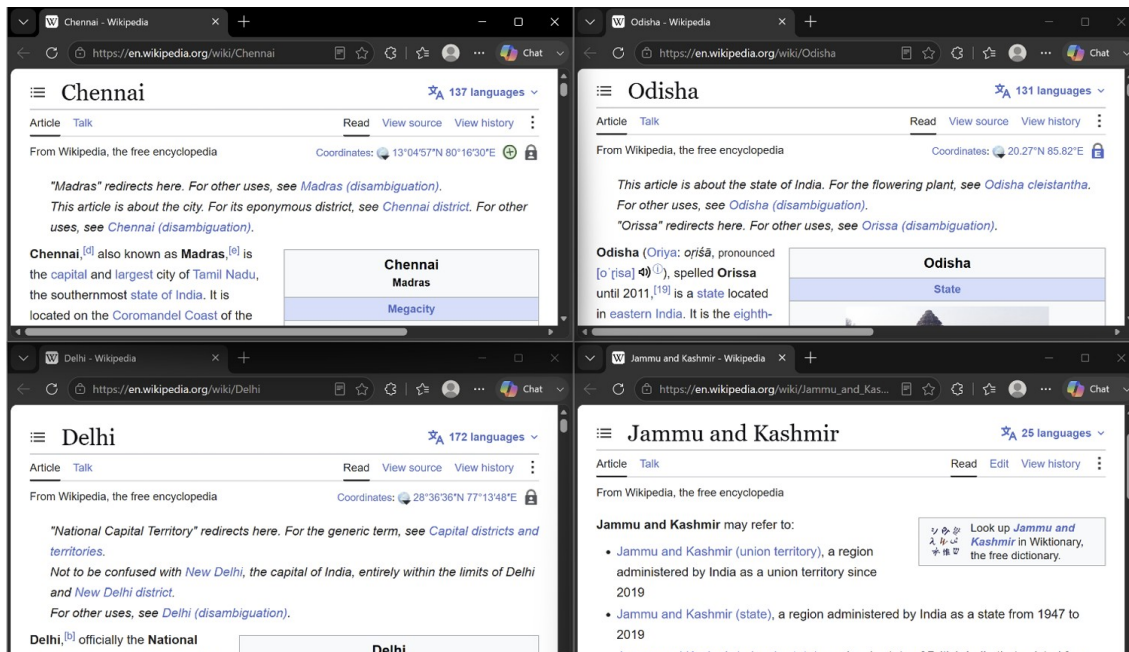
<!-- Chennai -->
<area
shape="circle"
coords="512,620,18"
href="https://en.wikipedia.org/wiki/Chennai"
target="_blank"
alt="Chennai">

</map>
</body>
</html>
```

CSS CODE:

```
body{
    margin:0;
    padding:20px;
    background:#f5f5f5;
    font-family:Arial, Helvetica, sans-serif;
    text-align:center;
}
h1{
    color:darkblue;
    margin-bottom:5px;
}
h2{
    color:crimson;
    margin-top:5px;
}
p{
    font-size:20px;
    color:#333;
}
.map{
    width:850px;
    max-width:100%;
    border:3px solid black;
    border-radius:8px;
    margin-top:20px;
    box-shadow:0 0 12px gray;
}
footer{
```

```
margin-top:25px;
color:#666;
font-size:16px;
}
```

OUTPUT:**WIKIPEDIA PAGES IT OPENS TO:****RESULT:**

Thus, a webpage containing an India Map with clickable hotspots was successfully created using HTML Image Maps (<map> and <area>) and CSS. Each hotspot correctly navigates to its corresponding Wikipedia page when clicked.

EX.NO.4	REGISTRATION FORM FOR AN APPLICATION
DATE: 20.07.26	

AIM

To design and develop a **PAN Card Registration Form** using **HTML and CSS** that collects applicant personal details, address details, contact details, identity details, and PAN application information

ALGORITHM

1. Create an HTML file with the basic structure using <html>, <head>, and <body> tags.
2. Add the page title and link the external CSS file using the <link> tag.
3. Design the webpage using an Indian Government (Income Tax Department) theme with appropriate colors and layout.
4. Create a <form> element to hold all PAN registration fields.
5. Add text fields for applicant name, father's name, mother's name, address, city, state, district, and PIN code using <input type="text">.
6. Add a date field for date of birth using <input type="date">.
7. Add radio buttons for gender and citizenship using <input type="radio">.
8. Add a drop-down list for state and identity proof using <select> and <option> tags.
9. Add fields for Aadhaar Number, Mobile Number, and Email ID using appropriate HTML input types.
10. Add a declaration checkbox confirming that the information provided is correct.
11. Add **Submit** and **Reset** buttons using <input type="submit"> and <input type="reset">.
12. Apply CSS styles to provide an official **Income Tax Department** appearance using Indian Government colors, borders, spacing, typography, and responsive layout.
13. Save both HTML and CSS files in the same folder and execute the HTML file in a web browser.
14. Verify that the PAN Card Registration Form is displayed correctly and accepts user input successfully.

CODE**HTML:**

```

<!DOCTYPE html>
<html>
<head>
  <title>PAN Card Registration</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h2>Government of India</h2>
    <h3>Income Tax Department</h3>
    <h1>PAN Card Registration Form</h1>
    <form>
      <label>Full Name</label>
      <input type="text" placeholder="Enter Name" required>
      <label>Father's Name</label>
      <input type="text" placeholder="Father's Name" required>
      <label>Date of Birth</label>

```

```

<input type="date" required>
<label>Gender</label>
<input type="radio" name="gender"> Male
<input type="radio" name="gender"> Female
<input type="radio" name="gender"> Other
<br><br>
<label>Aadhaar Number</label>
<input type="text" placeholder="12 Digit Aadhaar Number">
<label>Mobile Number</label>
<input type="tel" placeholder="10 Digit Mobile Number">
<label>Email</label>
<input type="email" placeholder="Email Address">
<label>Address</label>
<textarea placeholder="Enter Address"></textarea>
<label>State</label>
<select>
<option>Tamil Nadu</option>
<option>Kerala</option>
<option>Karnataka</option>
<option>Andhra Pradesh</option>
<option>Telangana</option>
</select>
<br><br>
<input type="checkbox">
I declare that the information given above is correct.

<br><br>
<input type="submit" value="Register">
<input type="reset" value="Reset">
</form>
</div>
</body>
</html>

```

CSS:

```

body{
  font-family:Arial,sans-serif;
  background:#eef4f9;
}
.container{
  width:450px;
  margin:40px auto;
  background:white;
  padding:25px;
  border-radius:10px;
  box-shadow:0 0 10px gray;
}
h1{
  color:#0b5394;
  text-align:center;
}
h2,h3{
  text-align:center;
  margin:5px;
}

```

```
}  
label{  
    display:block;  
    margin-top:12px;  
    font-weight:bold;  
}  
  
input[type=text],  
input[type=email],  
input[type=tel],  
input[type=date],  
textarea,  
select{  
    width:100%;  
    padding:8px;  
    margin-top:5px;  
    border:1px solid gray;  
    border-radius:5px;  
}  
  
input[type=reset]{  
    background:red;  
    color:white;  
    padding:10px 18px;  
    border:none;  
    border-radius:5px;  
    cursor:pointer;  
}  
  
input[type=submit]:hover{  
    background:#073763;  
}  
  
input[type=reset]:hover{  
    background:darkred;  
}
```

OUTPUT

Apply for New PAN Card

Fill all mandatory details carefully before submitting your PAN Card Application.

Applicant Name Enter Full Name	Date of Birth mm/dd/yyyy
Father's Name Enter Father's Name	Mother's Name Enter Mother's Name
Gender ☒ Male ☐ Female ☐ Other	Citizen of India ☒ Yes ☐ No
Aadhaar Number 12 Digit Aadhaar Number	Mobile Number 10 Digit Mobile Number
Email Address example@gmail.com	Occupation Student

Residential Address

House / Flat Number House No.	Street / Area Street Name
City City	District District
State Select State	PIN Code PIN Code

Identity Verification

Proof of Identity Aadhaar Card	Proof of Address Aadhaar Card
Preferred Communication Email	Application Type New PAN

☐ I hereby declare that all the information provided by me is true and correct to the best of my knowledge. I understand that submitting false information may result in rejection of my PAN Card application.

Apply for New PAN Card

Fill all mandatory details carefully before submitting your PAN Card Application.

Applicant Name Ajay.I	Date of Birth 06 / 22 / 2007
Father's Name IYANRAJ	Mother's Name XXXXXXXX
Gender ☒ Male ☐ Female ☐ Other	Citizen of India ☒ Yes ☐ No
Aadhaar Number XXXXXXXX23456	Mobile Number 8124297703
Email Address ajayianraj.2401007@srec.ac.in	Occupation Student

Residential Address

House / Flat Number xxxx	Street / Area xxx
City coimbatore	District xxx
State Tamil Nadu	PIN Code xxxxxx

Identity Verification

Proof of Identity Aadhaar Card	Proof of Address Aadhaar Card
Preferred Communication Email	Application Type New PAN

☒ I hereby declare that all the information provided by me is true and correct to the best of my knowledge. I understand that submitting false information may result in rejection of my PAN Card application.

Submit Application Reset

✓

PAN Card Registration Successful

Your application has been submitted successfully.

Application Number : PAN-ROBFT-893051

Applicant Name : Ajay.I

Mobile Number : 8124297703

Please save your PAN Application Number for future reference.

Submit Another Application

RESULT

Thus, a **PAN Card Registration Form** based on the **Government of India Income Tax Department** theme was successfully designed and developed using **HTML** and **CSS**. The webpage successfully accepts applicant details, validates user inputs, and displays a confirmation page upon successful registration.

EX.NO.5	COLLEGE WEBPAGE CREATION USING HTML & CSS
DATE: 27.07.26	

AIM:

To design and develop a multi-page college website using HTML and CSS, featuring a common navigation menu.

ALGORITHM:

1. Start.
2. Create a project folder and place the college emblem image inside it.
3. Design a common header section containing the college emblem and name, to be reused across all pages.
4. Create a navigation bar with menu options: Home, Events, Admission, Fees, Results and Contact Us.
5. Link each menu option to its corresponding HTML page using the tag.
6. Design the Home page (index.html) with a hero section that prominently displays the college emblem, a welcome message and an Apply Now button.
7. Design the Events page to list upcoming college events using card layouts.
8. Design the Admission page with eligibility details and an Enquire Now button.
9. Design the Fees page with a table showing the fee structure of different programs.
10. Design the Results page with a table showing sample semester results.
11. Design the Contact Us page with the college address, phone, email and a feedback form.
12. Create an external stylesheet (style.css) and link it to every page to maintain a consistent theme (colors, fonts, spacing) across the site.
13. Save all files with proper extensions (.html, .css) in the same folder.
14. Open index.html in a web browser and verify that every navigation link correctly opens its respective page, and that the emblem appears on every page.
15. Stop.

CODE:**HTML:**

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>SRM University Thiruchiraapalli - Home</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<header>
<div class="brand">

<div class="brand-text">
<h1>SRM UNIVERSITY THIRUCHIRAAPALLI</h1>
<p>Deemed to be University</p>
</div>
</div>
</header>
<nav>
<ul>
<li><a href="index.html" class="active">Home</a></li>
```

```

        <li><a href="events.html">Events</a></li>
        <li><a href="admission.html">Admission</a></li>
        <li><a href="fees.html">Fees</a></li>
        <li><a href="results.html">Results</a></li>
        <li><a href="contact.html">Contact Us</a></li>
    </ul>
</nav>
</header>
<section class="hero">
    
    <h2>Welcome to SRM University Thiruchiraapalli</h2>
    <p class="tagline">Claim Your Future in Engineering & Beyond</p>
    <a href="admission.html" class="btn">Apply Now</a>
</section>
<main>
    <h2 class="section-title">About the College</h2>
    <p>SRM University Thiruchiraapalli is a multi-campus, multi-disciplinary
    research university committed to academic excellence, innovation and
    value-based education.</p>

    <h2 class="section-title" style="margin-top:40px;">Quick Links</h2>
    <div class="cards">
        <div class="card"><h3>Events</h3><p>Upcoming seminars, workshops and
    fests.</p></div>
        <div class="card"><h3>Admission</h3><p>Eligibility, entrance scores, how to
    apply.</p></div>
        <div class="card"><h3>Fees</h3><p>Semester-wise fee structure for every
    program.</p></div>
        <div class="card"><h3>Results</h3><p>Semester examination results.</p></div>
    </div>
</main>
<footer>&copy; 2026 SRM University Thiruchiraapalli. All Rights Reserved.</footer>
</body>
</html>

```

CSS:

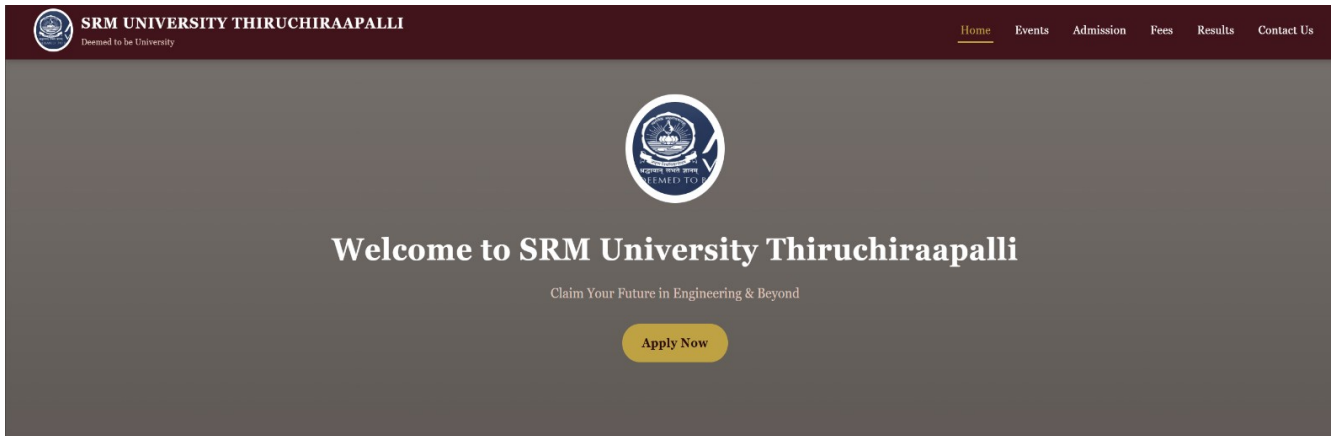
```

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
body {
    font-family: Georgia, 'Times New Roman', serif;
    background: #f3efe6;
    color: #26221f;
    line-height: 1.6;
}
header {
    background: #4a0e1a;
    color: #fff;
    display: flex;
    align-items: center;

```

```
justify-content: space-between;
padding: 10px 40px;
position: sticky;
top: 0;
z-index: 100;
box-shadow: 0 2px 8px rgba(0,0,0,0.25);
}
.brand {
  display: flex;
  align-items: center;
  gap: 12px;
}
.brand img.emblem {
  height: 55px;
  width: 55px;
  border-radius: 50%;
  background: #fff;
  padding: 3px;
}
.brand .brand-text h1 {
  font-size: 20px;
  letter-spacing: 1px;
  font-weight: 700;
}
.brand .brand-text p {
  font-size: 11px;
  letter-spacing: 0.5px;
  color: #e3c6ba;
}
nav ul {
  list-style: none;
  display: flex;
  gap: 26px;
}
nav ul li a {
  color: #fff;
  text-decoration: none;
  font-size: 15px;
  letter-spacing: 0.5px;
  padding: 6px 4px;
  border-bottom: 2px solid transparent;
  transition: border-color 0.25s ease, color 0.25s ease;
}
nav ul li a:hover,
nav ul li a.active {
  border-bottom: 2px solid #c9a227;
  color: #c9a227;
}
form {
  background: #fff;
  border: 1px solid #eadfd6;
```

```
border-radius: 6px;
padding: 30px;
max-width: 550px;
margin-top: 20px;
}
form label {
display: block;
margin-bottom: 6px;
font-weight: bold;
color: #4a0e1a;
}
form input, form textarea {
width: 100%;
padding: 10px;
margin-bottom: 18px;
border: 1px solid #d8c9bd;
border-radius: 4px;
font-family: inherit;
}
form button {
background: #4a0e1a;
color: #fff;
border: none;
padding: 12px 26px;
border-radius: 30px;
cursor: pointer;
font-size: 15px;
}
form button:hover {
background: #2f0812;
}
.contact-info {
margin-top: 20px;
}
.contact-info p {
margin-bottom: 8px;
}
/* ----- FOOTER ----- */
footer {
background: #1c0a10;
color: #e3c6ba;
text-align: center;
padding: 22px;
font-size: 13px;
}
```

OUTPUT:**About the College**

SRM University Thiruchiraapalli is a multi-campus, multi-disciplinary research university committed to academic excellence, innovation and value-based education. The institution offers a wide range of undergraduate, postgraduate and doctoral programs across engineering, medicine, management and the sciences.

Quick Links**Events**

Explore upcoming seminars, workshops and fests happening on campus.

Admission

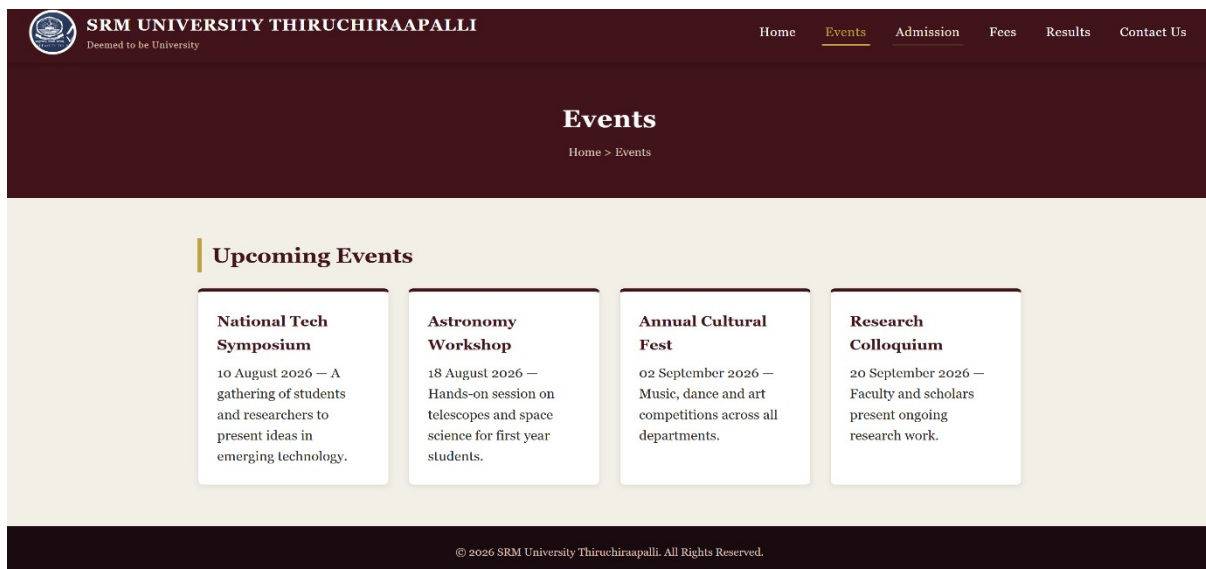
Learn about eligibility, entrance scores and how to apply.

Fees

View the semester-wise fee structure for every program.

Results

Check semester examination results and academic performance.




SRM UNIVERSITY THIRUCHIRAPPALLI
Deemed to be University

[Home](#)
[Events](#)
[Admission](#)
[Fees](#)
[Results](#)
[Contact Us](#)

Fee Structure

[Home > Fees](#)

Annual Fee Structure (2026-27)

Program	Tuition Fee / Year	Hostel Fee / Year
B.Tech (All Branches)	₹ 2,10,000	₹ 1,10,000
M.Tech	₹ 1,20,000	₹ 1,10,000
MBA	₹ 2,50,000	₹ 1,10,000
Ph.D	₹ 60,000	₹ 1,10,000

* Fees are indicative and subject to revision by the university administration.


SRM UNIVERSITY THIRUCHIRAPPALLI
Deemed to be University

[Home](#)
[Events](#)
[Admission](#)
[Fees](#)
[Results](#)
[Contact Us](#)

Get in Touch

Address: SRM University Thiruchirappalli, Tiruchirappalli, Tamil Nadu, India
Phone: +91 422 265 6422
Email: info@srmtiruchy.edu.in

Send a Message

Name

Email

Message

RESULT:

The college website was successfully designed and developed using HTML and CSS.

EX.NO.6	SHOPPING WEBPAGE USING CSS (NAVIGATION BAR AND DROP-DOWNS)
DATE: 03.08.26	

AIM:

To design a shopping webpage using HTML and CSS with a navigation bar and drop-down menus.

ALGORITHM:

1. Create the HTML structure.
2. Design a navigation bar with menu items.
3. Add drop-down menus for product categories.
4. Create product cards.
5. Style the webpage using CSS.
6. Apply colors, hover effects, and responsive layout.
7. Run the webpage in a browser.

CODE:**HTML:**

```
<!DOCTYPE html>
<html>
<head>
<title>Fresh Mart</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="nav">
<h2>Fresh Mart</h2>
<ul>
<li><a href="#">Home</a></li>
<li><a href="#">Products</a>
<ul>
<li><a href="#">Groceries</a></li>
<li><a href="#">Snacks</a></li>
<li><a href="#">Dairy Products</a></li>
<li><a href="#">Vegetables</a></li>
<li><a href="#">Fruits</a></li>
</ul>
</li>
<li><a href="#">Offers</a></li>
<li><a href="#">Contact</a></li>
</ul>
</div>
<div class="banner">
<h1>Welcome to Fresh Mart</h1>
<p>Fresh and Quality Products at Best Prices</p>
<button>Shop Now</button>
</div>
<div class="products">
<div class="card">
<h3>🥬 Vegetables</h3>
```

```

<p>Fresh farm vegetables.</p>
</div>
<div class="card">
<h3>🍎 Fruits</h3>
<p>Seasonal fresh fruits.</p>
</div>
<div class="card">
<h3>🥛 Dairy</h3>
<p>Milk, Butter and Cheese.</p>
</div>
<div class="card">
<h3>🍪 Snacks</h3>
<p>Biscuits, Chips and More.</p>
</div>
</div>
</body>
</html>

```

CSS:

```

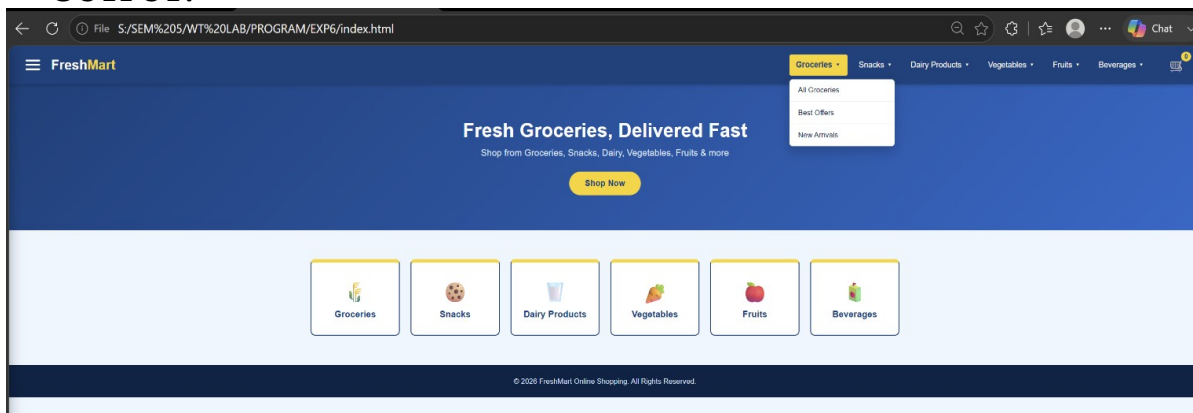
body{
margin:0;
font-family:Arial;
background:#eef6ff;
}
.nav{
background:#0066cc;
display:flex;
justify-content:space-between;
padding:10px 30px;
}
.nav h2{
color:yellow;
}
.nav ul{
list-style:none;
margin:0;
padding:0;
display:flex;
}
.nav ul li{
position:relative;
}
.nav ul li a{
display:block;
padding:15px;
color:white;
text-decoration:none;
}
.nav ul li:hover{
background:gold;
}
.nav ul li:hover>a{

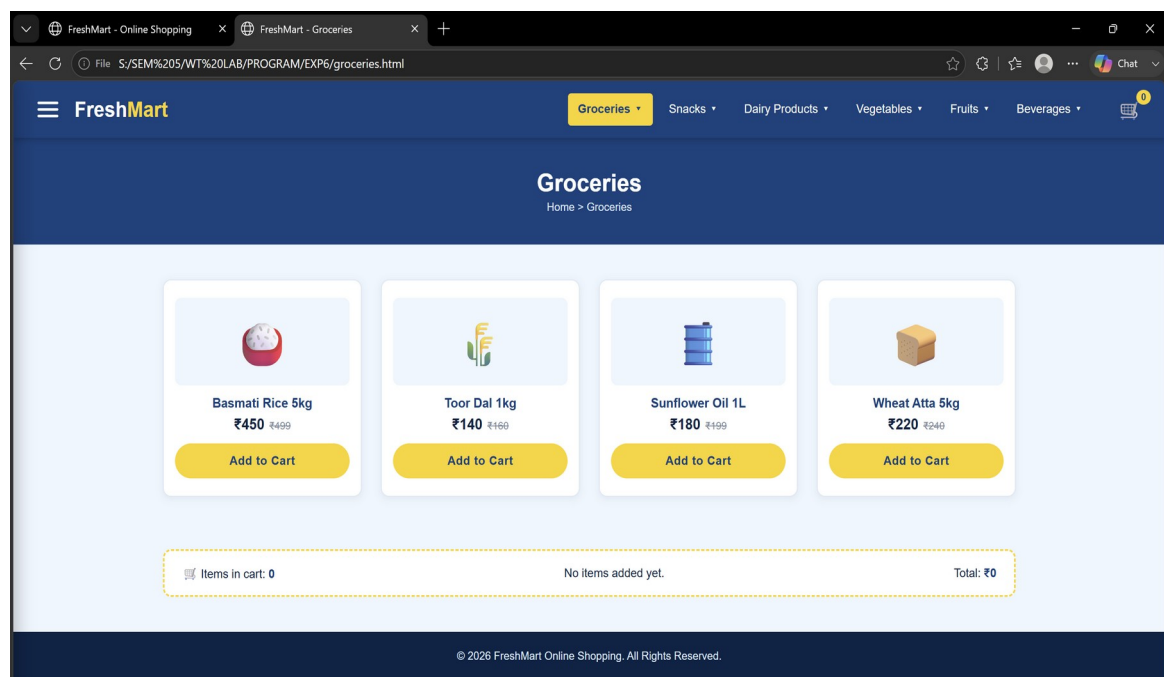
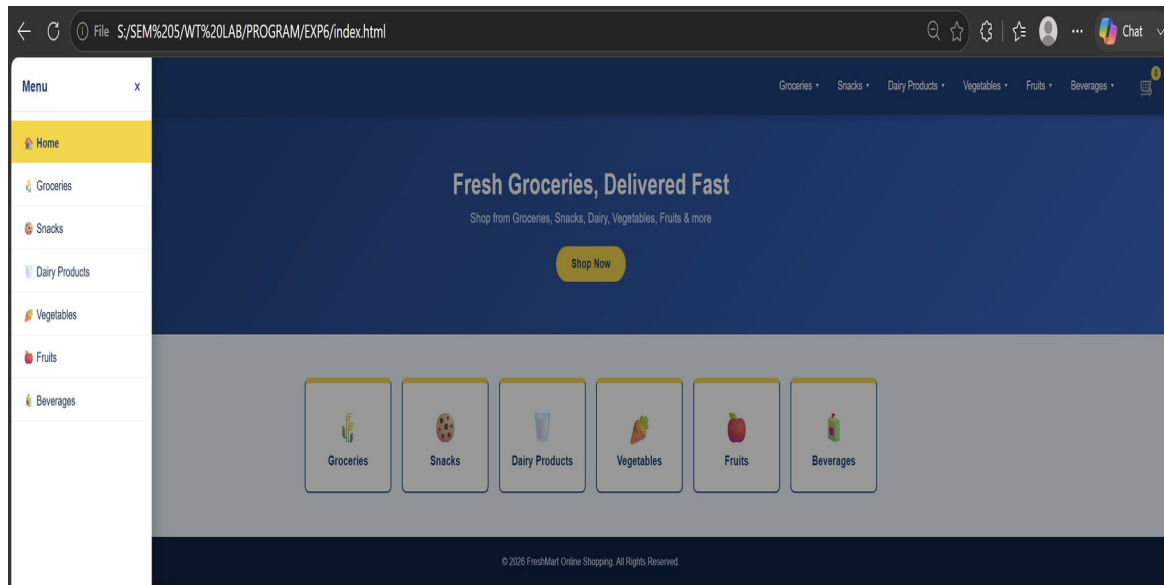
```

```

color:#003366;
}
.nav ul li ul{
display:none;
position:absolute;
background:white;
min-width:150px;
}
.nav ul li:hover ul{
display:block;
}
.nav ul li ul li{
display:block;
}
.nav ul li ul li a{
color:#003366;
}
.box{
text-align:center;
margin:80px;
padding:40px;
background:#0099ff;
color:white;
border-radius:10px;
}

```

OUTPUT:**Down Menu:****Drop**

Navigation Bar:**RESULT:**

A shopping webpage with an attractive navigation bar, drop-down menus, and product categories was created successfully using HTML and CSS.

EX.NO.7(a)	ELECTRICITY BILL CALCULATION
DATE: 24.08.26	

AIM:

To write a JavaScript program using conditional statements to calculate the electricity bill based on the number of units consumed.

ALGORITHM:

1. Start the program.
2. Read the number of units consumed.
3. If units are less than or equal to 100, calculate the bill at ₹2 per unit.
4. If units are between 101 and 200, calculate ₹2 for the first 100 units and ₹3 for the remaining units.
5. If units are above 200, calculate ₹2 for the first 100 units, ₹3 for the next 100 units, and ₹5 for the remaining units.
6. Display the total units consumed and the total electricity bill.
7. Stop the program.

PROGRAM:

```
let units = 250;
let bill;
if (units <= 100) {
    bill = units * 2;
}
else if (units <= 200) {
    bill = (100 * 2) + ((units - 100) * 3);
}
else {
    bill = (100 * 2) + (100 * 3) + ((units - 200) * 5);
}
console.log("Total Units Consumed: " + units);
console.log("Total Electricity Bill: ₹" + bill);
Output
Total Units Consumed: 250
Total Electricity Bill: ₹950
```

OUTPUT:

```
S:\SEM 5\WT LAB\PROGRAM\EXP7>node EXP7A.js
Total Units Consumed: 250
Total Electricity Bill: ₹750
```

RESULT:

Thus, the JavaScript program to calculate the electricity bill based on the units consumed was successfully executed.

EX.NO.7(b)	STUDENT MARKS ANALYSIS
DATE: 24.08.26	

AIM:

To write a JavaScript program using an array and loop to calculate the total, average, highest, and lowest marks of a student in five subjects.

ALGORITHM:

1. Start the program.
2. Store the marks of five subjects in an array.
3. Initialize the total as zero.
4. Use a loop to calculate the total marks.
5. Calculate the average by dividing the total by the number of subjects.
6. Find the highest and lowest marks using the Math.max() and Math.min() functions.
7. Display the total, average, highest mark, and lowest mark.
8. Stop the program.

PROGRAM:

```
let marks = [85, 78, 92, 88, 76];
let total = 0;
for (let i = 0; i < marks.length; i++) {
    total += marks[i];
}
let average = total / marks.length;
let highest = Math.max(...marks);
let lowest = Math.min(...marks);
console.log("Marks: " + marks);
console.log("Total Marks: " + total);
console.log("Average Marks: " + average);
console.log("Highest Mark: " + highest);
console.log("Lowest Mark: " + lowest);
```

Output

Marks: 85,78,92,88,76
Total Marks: 419
Average Marks: 83.8
Highest Mark: 92
Lowest Mark: 76

OUTPUT:

```
S:\SEM 5\WT LAB\PROGRAM\EXP7>node EXP7B.js
Enter mark for Subject 1: 86
Enter mark for Subject 2: 97
Enter mark for Subject 3: 56
Enter mark for Subject 4: 89
Enter mark for Subject 5: 99

Marks: 86,97,56,89,99
Total Marks: 427
Average Marks: 85.4
Highest Mark: 99
Lowest Mark: 56
```

RESULT:

Thus, the JavaScript program to analyse the marks of a student was successfully executed.

EX.NO.7(c)	ATM PIN VERIFICATION
DATE: 24.08.26	

AIM:

To write a JavaScript program using a loop to verify an ATM PIN with a maximum of three attempts.

ALGORITHM:

1. Start the program.
2. Store the correct PIN.
3. Allow the user to enter the PIN up to three times.
4. Compare the entered PIN with the correct PIN.
5. If the PIN is correct, display "PIN Verified" and stop.
6. If the PIN is incorrect, display "Wrong PIN Attempt".
7. After three incorrect attempts, display "ATM Blocked".
8. Stop the program.

PROGRAM:

```
let correctPIN = "2920";
let attempts = 0;
let verified = false;
while (attempts < 3) {
    let enteredPIN = prompt("Enter your PIN:");
    attempts++;
    if (enteredPIN === correctPIN) {
        console.log("PIN Verified");
        verified = true;
        break;
    }
    else {
        console.log("Wrong PIN Attempt");
    }
}
if (!verified) {
    console.log("ATM Blocked");
}
```

Output

For a correct PIN:

PIN Verified

For three wrong attempts:

Wrong PIN Attempt

Wrong PIN Attempt

Wrong PIN Attempt

ATM Blocked

OUTPUT:

```
S:\SEM 5\WT LAB\PROGRAM\EXP7>node EXP7C.js
Enter your PIN: 1220
Wrong PIN Attempt
Enter your PIN: 2920
PIN Verified
```

RESULT:

Thus, the JavaScript program to verify an ATM PIN with a maximum of three attempts was successfully executed.

EX.NO.7(d)	SHOPPING PURCHASE AMOUNT
DATE: 24.08.26	

AIM:

To write a JavaScript program using an array to calculate the total purchase amount of products selected by a customer.

ALGORITHM:

1. Start the program.
2. Store the prices of the selected products in an array.
3. Initialize the total amount as zero.
4. Use a loop to add all the product prices.
5. Check whether the total amount satisfies the given condition.
6. Display the appropriate message.
7. Stop the program.

PROGRAM:

```
let prices = [500, 750, 250, 1000];
let total = 0;
for (let i = 0; i < prices.length; i++) {
    total += prices[i];
}
console.log("Product Prices: " + prices);
console.log("Total Purchase Amount: ₹" + total);
if (total >= 2000) {
    console.log("Eligible for Discount");
}
else {
    console.log("Not Eligible for Discount");
}
Output
Product Prices: 500,750,250,1000
Total Purchase Amount: ₹2500
Eligible for Discount
```

OUTPUT:

```
S:\SEM 5\WT LAB\PROGRAM\EXP7>node EXP7D.js
Enter price of Product 1: 100
Enter price of Product 2: 800
Enter price of Product 3: 200
Enter price of Product 4: 134
Product Prices: 100,800,200,134
Total Purchase Amount: ₹1234
Not Eligible for Discount

S:\SEM 5\WT LAB\PROGRAM\EXP7>node EXP7D.js
Enter price of Product 1: 567
Enter price of Product 2: 230
Enter price of Product 3: 1456
Enter price of Product 4: 86
Product Prices: 567,230,1456,86
Total Purchase Amount: ₹2339
Eligible for Discount
```

RESULT:

Thus, the JavaScript program to calculate the total purchase amount using an array was successfully executed.

EX.NO.7(e)	EMPLOYEE ANNUAL SALARY
DATE: 24.08.26	

AIM:

To write a JavaScript program using an array of objects to calculate and display the annual salary of each employee.

ALGORITHM:

1. Start the program.
2. Store the employee name and monthly salary using an array of objects.
3. Use a loop to access each employee's details.
4. Calculate the annual salary using:

$$\text{Annual Salary} = \text{Monthly Salary} \times 12$$
5. Display the employee name, monthly salary, and annual salary.
6. Stop the program.

PROGRAM:

```
let employees = [
  { name: "Arun", salary: 25000 },
  { name: "Bala", salary: 30000 },
  { name: "Charan", salary: 28000 }
];
for (let i = 0; i < employees.length; i++) {
  let annualSalary = employees[i].salary * 12;
  console.log("Employee Name: " + employees[i].name);
  console.log("Monthly Salary: ₹" + employees[i].salary);
  console.log("Annual Salary: ₹" + annualSalary);
  console.log("-----");
}
```

OUTPUT:

```
S:\SEM 5\WT LAB\PROGRAM\EXP7>node EXP7E.js
Enter employee name: AJAY I
Enter monthly salary: 100000
Enter employee name: BHAVNA S
Enter monthly salary: 100001
Enter employee name: ASHWINN
Enter monthly salary: 200000

Employee Details
-----
Employee Name: AJAY I
Monthly Salary: ₹100000
Annual Salary: ₹1200000
-----
Employee Name: BHAVNA S
Monthly Salary: ₹100001
Annual Salary: ₹1200012
-----
Employee Name: ASHWINN
Monthly Salary: ₹200000
Annual Salary: ₹2400000
-----
```

RESULT:

Thus, the JavaScript program to calculate and display the annual salary of each employee was successfully executed.

EX.NO.8a**DATE: 07.09.26****JAVASCRIPT DOM (HOVER AND CLICKS ON A TABLE)****AIM:**

To implement JavaScript mouse events such as mouseover, mouseout, click and dblclick to dynamically modify the style of table rows.

ALGORITHM:

1. Create an HTML table with multiple rows.
2. Use mouseover to change the row background color to gray.
3. Use mouseout to return the row to its normal color.
4. Use click to toggle the gray background color.
5. Use dblclick to reset the row to its original style.
6. Display the changes dynamically without reloading the webpage.

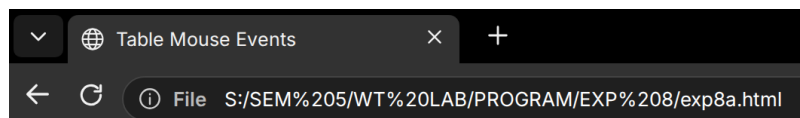
CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>Table Mouse Events</title>
  <style>
    table {
      border-collapse: collapse;
      width: 500px;
    }
    th, td {
      border: 1px solid black;
      padding: 10px;
    }
    th {
      background: lightblue;
    }
    .gray {
      background: gray;
    }
  </style>
</head>
<body>
  <h2>Table Mouse Events</h2>
  <table id="myTable">
    <tr>
      <th>Name</th>
      <th>Department</th>
    </tr>
    <tr>
      <td>Arun</td>
      <td>CSE</td>
    </tr>
    <tr>
      <td>Rahul</td>
```

```

        <td>ECE</td>
    </tr>
    <tr>
        <td>Kavi</td>
        <td>CHEM</td>
    </tr>
</table>
<script>
    let rows = document.querySelectorAll("#myTable tr");
    rows.forEach(function(row, index) {
        if (index != 0) {
            row.onmouseover = function() {
                row.style.backgroundColor = "gray";
            };
            row.onmouseout = function() {
                if (!row.classList.contains("gray"))
                    row.style.backgroundColor = "";
            };
            row.onclick = function() {
                row.classList.toggle("gray");
            };
            row.ondblclick = function() {
                row.classList.remove("gray");
                row.style.backgroundColor = "";
            };
        }
    });
</script>
</body>
</html>

```

OUTPUT:**Table Mouse Events**

Name	Department
Ajay	CSE
Bhavna	CA
Kabiles	CHEM

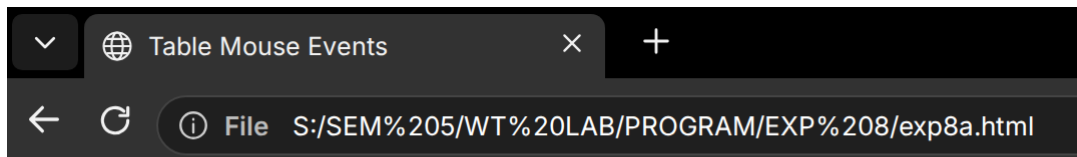


Table Mouse Events

Name	Department
Ajay	CSE
Bhavna	CA
Kabilesh	CHEM

RESULT:

Thus, the JavaScript mouse events mouseover, mouseout, click and dblclick were successfully implemented on the HTML table.

EX.NO.8b	JAVASCRIPT DOM (REAL-TIME DIGITAL CLOCK)
DATE: 07.09.26	

AIM:

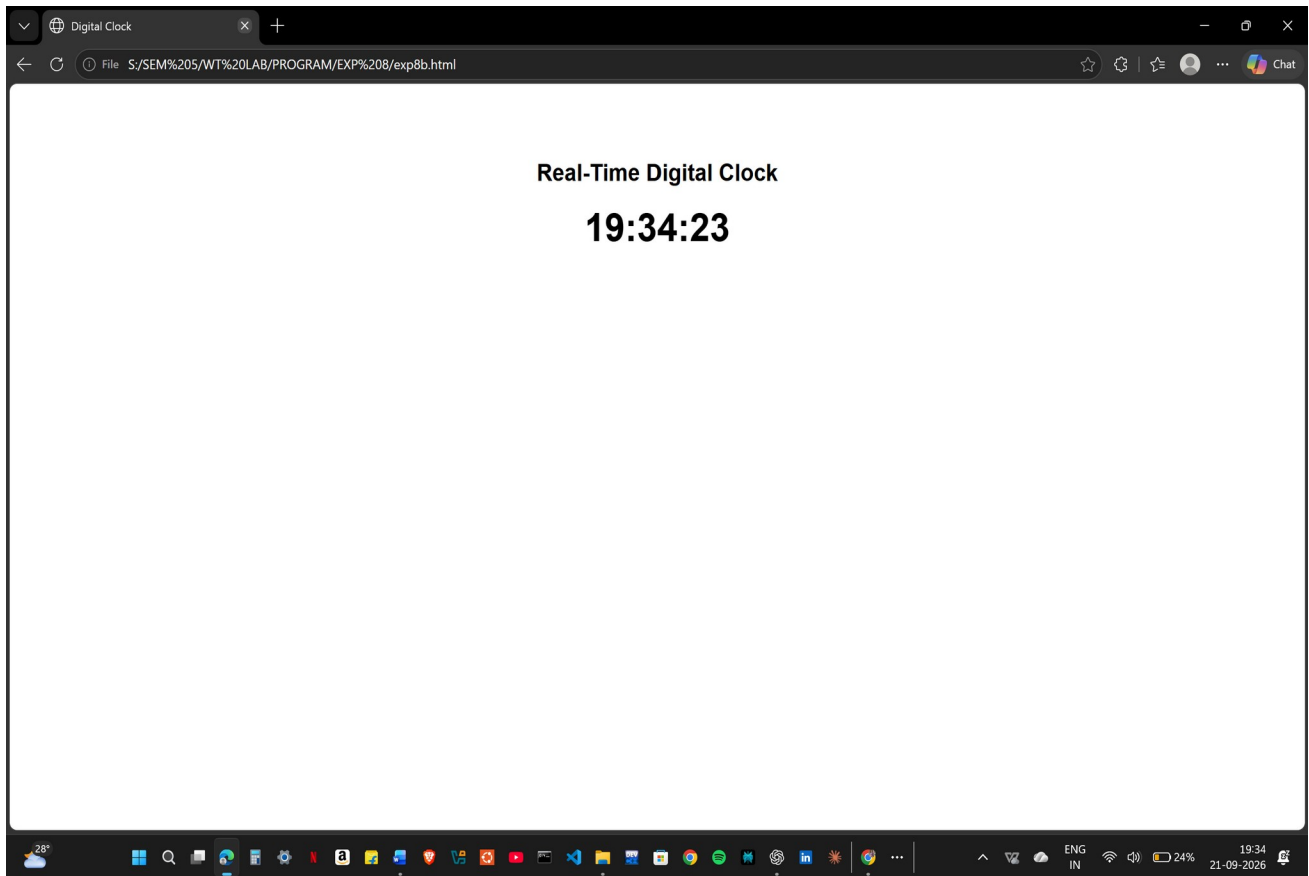
To create a real-time digital clock using JavaScript that displays the current time and updates every second using setInterval().

ALGORITHM:

1. Create an empty HTML element with the ID clock.
2. Create a JavaScript function named showTime().
3. Create a new Date object to obtain the current time.
4. Extract hours, minutes and seconds.
5. Display the time in hh:mm:ss format.
6. Use setInterval() to update the clock every second.

CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>Digital Clock</title>
  <style>
    body {
      text-align: center;
      font-family: Arial;
      padding: 50px;
    }
    #clock {
      font-size: 40px;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <h2>Real-Time Digital Clock</h2>
  <div id="clock"></div>
  <script>
    function showTime() {
      let now = new Date();
      let h = String(now.getHours()).padStart(2, "0");
      let m = String(now.getMinutes()).padStart(2, "0");
      let s = String(now.getSeconds()).padStart(2, "0");
      document.getElementById("clock").textContent =
        h + ":" + m + ":" + s;
    }
    showTime();
    setInterval(showTime, 1000);
  </script>
</body>
</html>
```

OUTPUT:**RESULT:**

Thus, a real-time digital clock was successfully created using JavaScript Date() and setInterval().

EX.NO.8c**JAVASCRIPT DOM (KEYBOARD INTERACTION)****DATE: 07.09.26****AIM:**

To implement keyboard interaction in JavaScript to change and reset the border of an HTML table using keyboard keys.

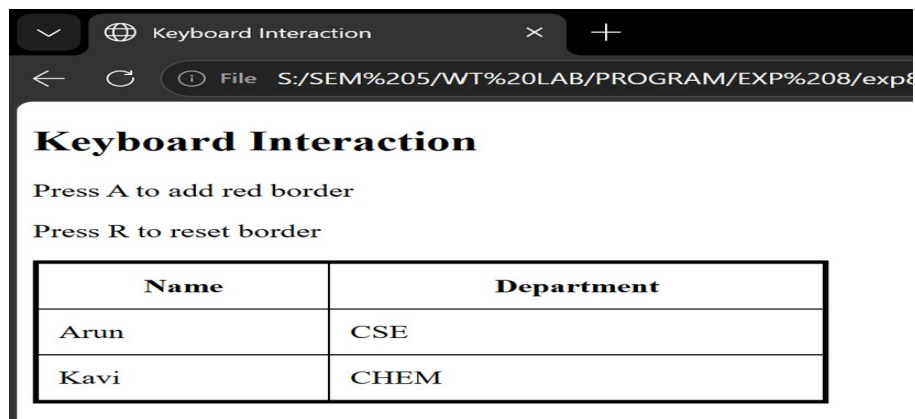
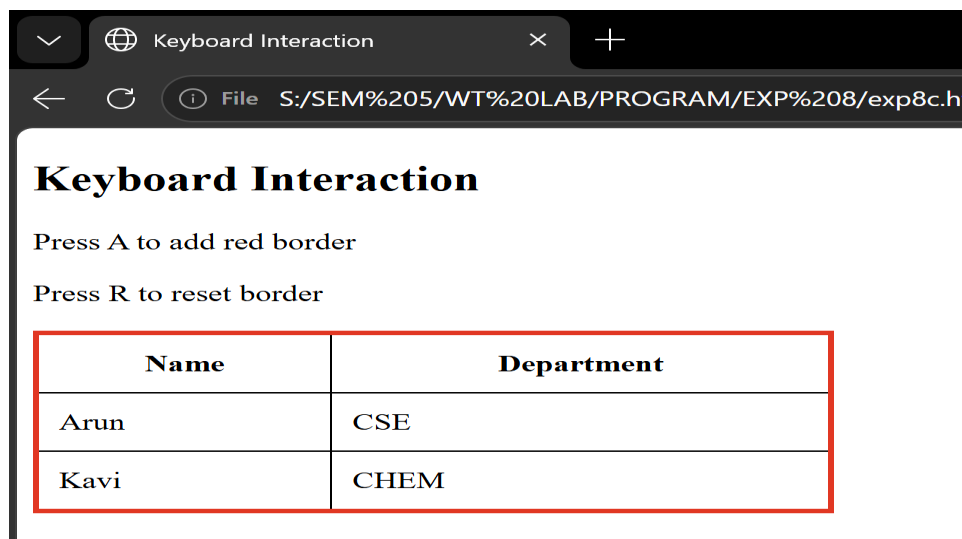
ALGORITHM:

1. Create an HTML table.
2. Add a keyboard event listener using keydown.
3. Check whether the pressed key is A.
4. If the key is A, add a red border to the table.
5. If the key is R, reset the table border to black.
6. Display the changes dynamically.

CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>Keyboard Interaction</title>
  <style>
    table {
      border-collapse: collapse;
      width: 400px;
      border: 3px solid black;
    }
    th, td {
      border: 1px solid black;
      padding: 10px;
    }
  </style>
</head>
<body>
<h2>Keyboard Interaction</h2>
<p>Press A to add red border</p>
<p>Press R to reset border</p>
<table id="myTable">
  <tr>
    <th>Name</th>
    <th>Department</th>
  </tr>
  <tr>
    <td>Arun</td>
    <td>CSE</td>
  </tr>
  <tr>
    <td>Kavi</td>
    <td>CHEM</td>
  </tr>
</table>
<script>
```

```
document.addEventListener("keydown", function(e) {  
  if (e.key.toLowerCase() === "a") {  
    document.getElementById("myTable").style.border =  
      "3px solid red";  
  }  
  if (e.key.toLowerCase() === "r") {  
    document.getElementById("myTable").style.border =  
      "3px solid black";  
  }  
});  
</script>  
</body>  
</html>
```

OUTPUT:**RESULT:**

Thus, keyboard interaction was successfully implemented using the JavaScript keydown event.

EX.NO.8d	JAVASCRIPT DOM (SIMPLE FORM WITH DISPLAY)
DATE: 07.09.26	

AIM:

To create a simple HTML form and display the user-entered value dynamically using JavaScript without reloading the webpage.

ALGORITHM:

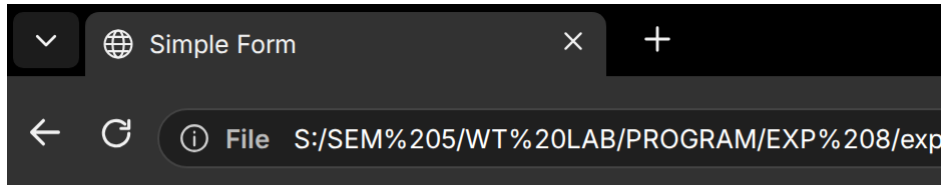
1. Create a form containing a text field and submit button.
2. Add a paragraph element to display the entered value.
3. Add a submit event listener to the form.
4. Use e.preventDefault() to prevent page reload.
5. Read the input value using the value property.
6. Display the value inside the paragraph using textContent.

CODE:

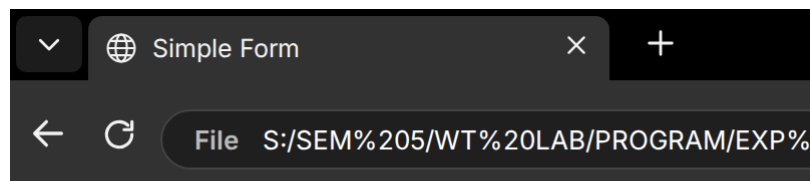
```
<!DOCTYPE html>
<html>
<head>
  <title>Simple Form</title>
  <style>
    body {
      font-family: Arial;
      padding: 30px;
    }
    input {
      padding: 10px;
    }
    button {
      padding: 10px;
    }
    p {
      font-weight: bold;
    }
  </style>
</head>
<body>
<h2>Simple Form</h2>
<form id="myForm">
  <input type="text" id="name"
    placeholder="Enter your name" required>
  <button type="submit">Submit</button>
</form>
<p id="output"></p>
<script>
  document.getElementById("myForm").addEventListener(
    "submit",

    function(e) {
      e.preventDefault();
      let value = document.getElementById("name").value;
```

```
document.getElementById("output").textContent =  
    "You entered: " + value;  
}  
);  
</script>  
</body>  
</html>
```

OUTPUT:

Simple Form

Simple Form

You entered: Ajay I

RESULT:

Thus, a simple form was successfully created and the entered value was displayed using JavaScript without reloading the webpage.

EX.NO.8e	JAVASCRIPT DOM (MINI LOGIN SYSTEM)
DATE: 07.09.26	

AIM:

To implement a basic login authentication system using JavaScript by validating username and password.

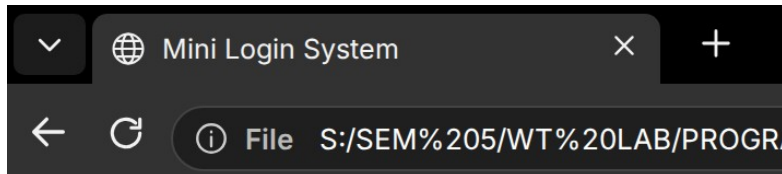
ALGORITHM:

1. Create username and password input fields.
2. Create a Login button.
3. Read the values entered by the user.
4. Compare the username with admin.
5. Compare the password with 1234.
6. If both are correct, display Welcome.
7. Otherwise, display Invalid credentials.

CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>Mini Login System</title>
  <style>
    body {
      font-family: Arial;
      padding: 30px;
    }
    input {
      display: block;
      margin: 10px 0;
      padding: 10px;
    }
    button {
      padding: 10px 20px;
    }
    #message {
      font-weight: bold;
    }
  </style>
</head>
<body>
  <h2>Mini Login System</h2>
  <input type="text" id="username" placeholder="Username">
  <input type="password" id="password" placeholder="Password">
  <button onclick="login()">Login</button>
  <p id="message"></p>
  <script>
    function login() {
      let username = document.getElementById("username").value;
      let password = document.getElementById("password").value;
      if (username === "admin" && password === "1234") {
        document.getElementById("message").textContent =
```

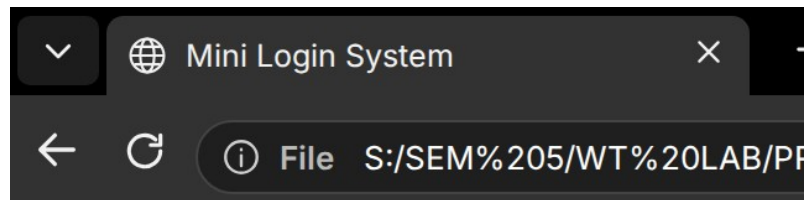
```
        "Welcome";  
    } else {  
        document.getElementById("message").textContent =  
            "Invalid credentials";  
    }  
}  
</script>  
</body>  
</html>
```

OUTPUT:

Mini Login System



Login



Mini Login System



Welcome

RESULT:

Thus, a basic login authentication system was successfully implemented using JavaScript

EX.NO.8f	JAVASCRIPT DOM (DARK/LIGHT MODE)
DATE: 07.09.26	

AIM:

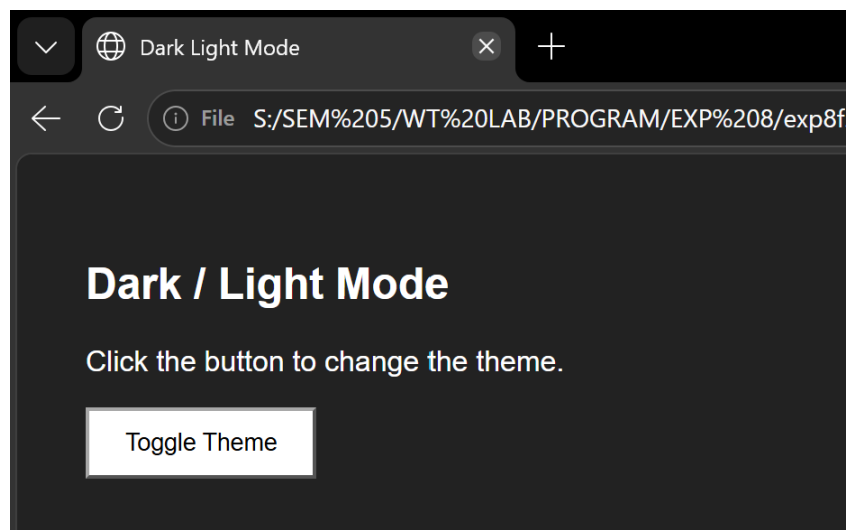
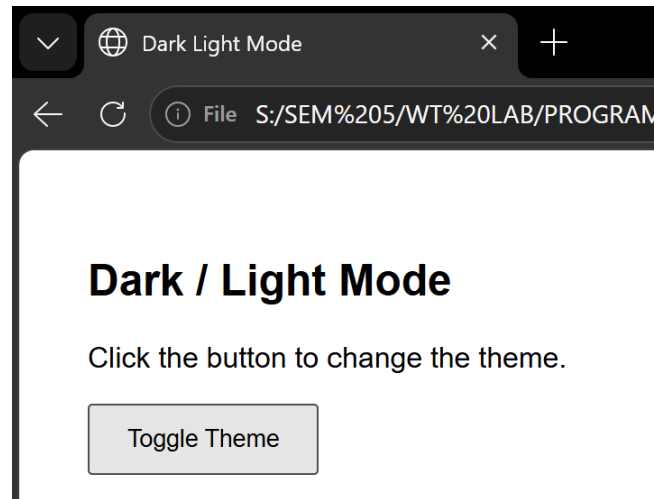
To dynamically change the webpage between dark mode and light mode using JavaScript and `classList.toggle()`.

ALGORITHM:

1. Create a webpage with a Toggle Theme button.
2. Define a `.dark-mode` CSS class.
3. Set a dark background and light text inside `.dark-mode`.
4. Add a click event to the Toggle Theme button.
5. Use `classList.toggle("dark-mode")` to add or remove the class.
6. Change the webpage theme dynamically without reloading.

CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>Dark Light Mode</title>
  <style>
    body {
      font-family: Arial;
      padding: 30px;
    }
    button {
      padding: 10px 20px;
    }
    .dark-mode {
      background: #222;
      color: white;
    }
    .dark-mode button {
      color: black;
    }
  </style>
</head>
<body>
  <h2>Dark / Light Mode</h2>
  <p>Click the button to change the theme.</p>
  <button id="themeBtn">Toggle Theme</button>
  <script>
    document.getElementById("themeBtn").onclick = function() {
      document.body.classList.toggle("dark-mode");
    };
  </script>
</body>
</html>
```

OUTPUT:**RESULT:**

Thus, the dark/light theme was successfully implemented using JavaScript DOM manipulation and `classList.toggle()`

EX.NO.9	ANGULAR JS FORM VALIDATION
DATE: 21.09.26	

AIM:

To design and implement a Student Registration Form using AngularJS form validation directives and validation properties.

ALGORITHM:

1. Create an HTML webpage containing a student registration form.
2. Include fields for Name, Email, Mobile Number and Password.
3. Include the AngularJS library in the webpage.
4. Use AngularJS form validation directives such as ng-model, required, ng-pattern and ng-minlength.
5. Validate the name field to ensure it is not empty.
6. Validate the email field using the email format.
7. Validate the mobile number to contain exactly 10 digits.
8. Validate the password to contain at least 6 characters.
9. Display suitable error messages when the entered data is invalid.
10. Disable the Register button until all fields are valid.
11. Display a successful registration message when valid details are submitted.

CODE:

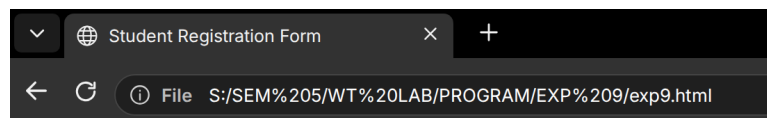
```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <title>Student Registration Form</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>
  <style>
    body {
      font-family: Arial;
      padding: 30px;
    }
    form {
      width: 400px;
      padding: 20px;
      border: 2px solid black;
    }
    input {
      width: 95%;
      padding: 10px;
      margin: 5px 0;
    }
    button {
      padding: 10px 20px;
      margin-top: 10px;
    }
    .error {
      color: red;
      font-size: 13px;
```

```

    }
    .success {
        color: green;
        font-weight: bold;
    }
</style>
</head>
<body ng-controller="formCtrl">
    <h2>Student Registration Form</h2>
    <form name="studentForm" ng-submit="register()" novalidate>
        <label>Name:</label>
        <input type="text"
            name="name"
            ng-model="student.name"
            required>
        <p class="error"
            ng-show="studentForm.name.$touched &&
                studentForm.name.$invalid">
            Name is required
        </p>
        <label>Email:</label>
        <input type="email"
            name="email"
            ng-model="student.email"
            required>
        <p class="error"
            ng-show="studentForm.email.$touched &&
                studentForm.email.$invalid">
            Enter a valid email address
        </p>
        <label>Mobile Number:</label>
        <input type="text"
            name="mobile"
            ng-model="student.mobile"
            ng-pattern="/^[0-9]{10}$/"
            required>
        <p class="error"
            ng-show="studentForm.mobile.$touched &&
                studentForm.mobile.$invalid">
            Mobile number must contain exactly 10 digits
        </p>
        <label>Password:</label>
        <input type="password"
            name="password"
            ng-model="student.password"
            ng-minlength="6"
            required>
        <p class="error"
            ng-show="studentForm.password.$touched &&
                studentForm.password.$invalid">
            Password must contain at least 6 characters

```

```
</p>
<button type="submit"
  ng-disabled="studentForm.$invalid">
  Register
</button>
</form>
<p class="success" ng-if="registered">
  Registration Successful!
</p>
<script>
  var app = angular.module("myApp", []);
  app.controller("formCtrl", function($scope) {
    $scope.student = {};
    $scope.register = function() {
      $scope.registered = true;
    };
  });
</script>
</body>
</html>
```

OUTPUT:**Student Registration Form**

Name:	Ajay I
Email:	ajay.2401007@srec.ac.in
Mobile Number:	9025467813
Password:	
Register	

Student Registration Form

Name:

Email:

Mobile Number:

Password:

Registration Successful!

RESULT:

Thus, the Student Registration Form using AngularJS was successfully implemented with form validation directives and validation properties.

EX.NO.10	REACT JS: COMPONENTS, PROPS, STATE AND EVENTS
DATE: 28.09.26	

AIM:

To develop a Student Profile Dashboard using React JS by implementing components, JSX, props, state, and event handling to dynamically display and update student information.

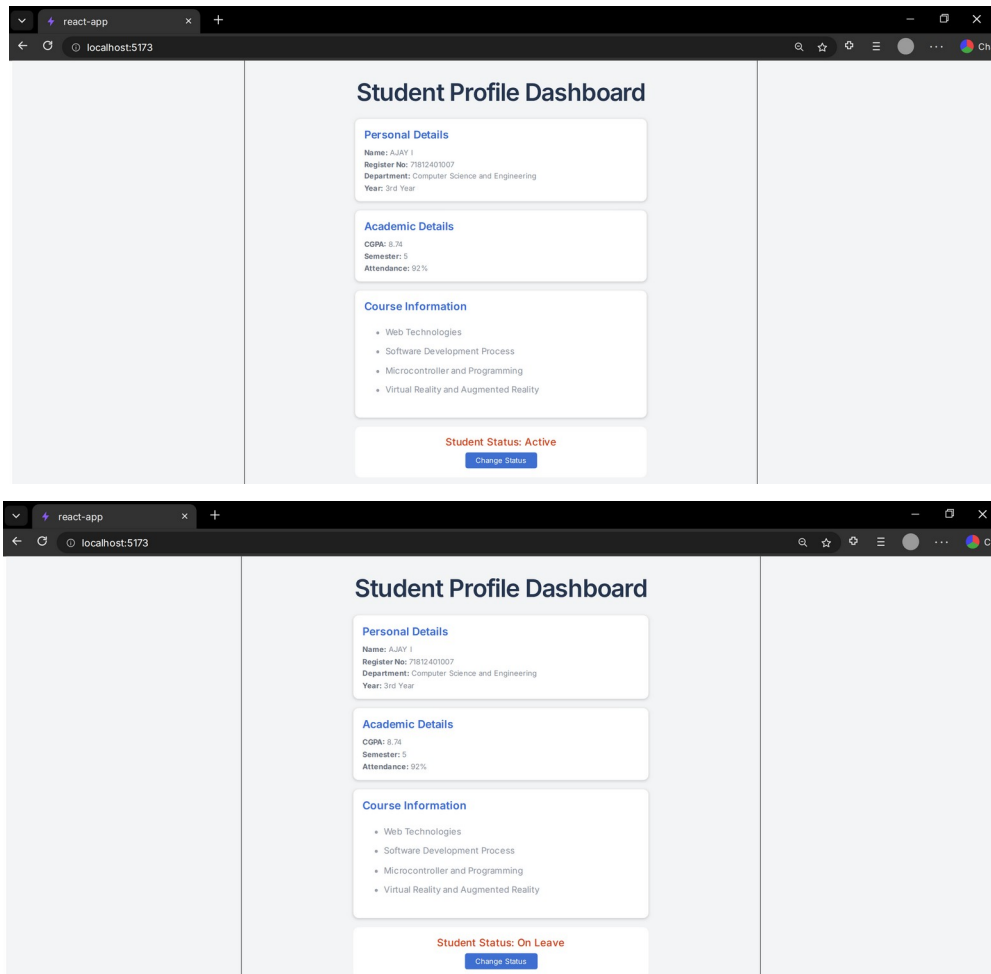
Algorithm

1. Create a new React JS application.
2. Create separate components for Personal Details, Academic Details, and Course Information.
3. Use JSX to design the Student Profile Dashboard.
4. Define student information in the main component.
5. Pass student details to child components using Props.
6. Use State to maintain the student's current status.
7. Implement event handling using buttons to change the student's status.
8. Apply CSS styling to improve the appearance of the dashboard.
9. Run the React application and verify the displayed student information.
10. Observe that the student status changes dynamically when the button is clicked.

CODE:

```
import React, { useState } from "react";
import "./App.css";
function PersonalDetails({ student }) {
  return (
    <div className="card">
      <h2>Personal Details</h2>
      <p><b>Name:</b> {student.name}</p>
      <p><b>Register No:</b> {student.regNo}</p>
      <p><b>Department:</b> {student.department}</p>
      <p><b>Year:</b> {student.year}</p>
    </div>
  );
}
function AcademicDetails({ student }) {
  return (
    <div className="card">
      <h2>Academic Details</h2>
      <p><b>CGPA:</b> {student.cgpa}</p>
      <p><b>Semester:</b> {student.semester}</p>
      <p><b>Attendance:</b> {student.attendance}%</p>
    </div>
  );
}
function CourseInformation({ courses }) {
  return (
    <div className="card">
      <h2>Course Information</h2>
      <ul>
        {courses.map((course, index) => (
          <li key={index}>{course}</li>
        ))}
      </ul>
    </div>
  );
}
```

```
    )}
  </ul>
</div>
);
}
function App() {
  const [status, setStatus] = useState("Active");
  const student = {
    name: "AJAY.I",
    regNo: "71812401007",
    department: "Computer Science and Engineering",
    year: "3rd Year",
    cgpa: "8.74",
    semester: "5",
    attendance: "92"
  };
  const courses = [
    "Mass Transfer Operations",
    "Chemical Reaction Engineering",
    "Process Control",
    "Heat Transfer"
  ];
  const changeStatus = () => {
    setStatus(status === "Active" ? "On Leave" : "Active");
  };
  return (
    <div className="container">
      <h1>Student Profile Dashboard</h1>
      <PersonalDetails student={student} />
      <AcademicDetails student={student} />
      <CourseInformation courses={courses} />
      <div className="status">
        <h2>Student Status: {status}</h2>
        <button onClick={changeStatus}>
          Change Status
        </button>
      </div>
    </div>
  );
}
export default
```

OUTPUT:**RESULT:**

Thus, the Student Profile Dashboard was successfully developed using React JS.

EX.NO.11	REACT JS: COMPONENT STATE AND INPUT HANDLING
DATE: 28.09.26	

AIM:

To develop a Course Registration System using React JS that accepts student details through form inputs, manages the values using component state, validates the required fields, and dynamically displays the registered course details after successful submission.

ALGORITHM:

1. Create a React JS application.
2. Create a registration form using JSX.
3. Create state variables to store the student's name, register number, email, department, and selected course.
4. Use onChange events to update the state whenever the user enters or changes a value.
5. Use onSubmit to handle the form submission.
6. Validate all required fields before submission.
7. Display an error message if any required field is empty.
8. If all fields are valid, store the submitted details in the component state.
9. Dynamically display the registered student's details and selected course.
10. Apply CSS styling to improve the appearance of the registration form.

CODE:

```
import React, { useState } from "react";
import "./App.css";
function App() {
  const [data, setData] = useState({
    name: "", regNo: "", email: "", department: "", course: ""
  });
  const [result, setResult] = useState(null);
  const change = (e) => {
    setData({ ...data, [e.target.name]: e.target.value });
  };
  const submit = (e) => {
    e.preventDefault();
    if (Object.values(data).some(x => !x)) {
      alert("Please fill all fields");
      return;
    }
    setResult(data);
  };
  return (
    <div className="container">
      <h1>Course Registration System</h1>
      <form onSubmit={submit}>
        <input name="name" placeholder="Student Name" onChange={change} />
        <input name="regNo" placeholder="Register Number" onChange={change} />
        <input name="email" type="email" placeholder="Email" onChange={change} />
        <select name="department" onChange={change}>
          <option value="">Select Department</option>
          <option>Chemical Engineering</option>
          <option>Computer Science</option>
        </select>
      </form>
    </div>
  );
}
```

```
      <option>Information Technology</option>
    </select>
    <select name="course" onChange={change}>
      <option value="">Select Course</option>
      <option>React JS</option>
      <option>Python</option>
      <option>Data Science</option>
      <option>Machine Learning</option>
    </select>
    <button>Register</button>
  </form>
  {result && (
    <div className="result">
      <h2>Registration Successful</h2>
      <p>Name: {result.name}</p>
      <p>Register No: {result.regNo}</p>
      <p>Email: {result.email}</p>
      <p>Department: {result.department}</p>
      <p>Course: {result.course}</p>
    </div>
  )}
</div>
);
}
export default App;
```

OUTPUT:

The screenshot displays a web application running on a browser at localhost:5173. The page features a 'Course Registration System' form. The form is a white card with a light blue border, centered on a light blue background. It contains the following elements:

- Student Name:** A text input field with the placeholder 'Enter your name'.
- Register Number:** A text input field with the placeholder 'Enter register number'.
- Email:** A text input field with the placeholder 'Enter email'.
- Department:** A dropdown menu with the placeholder 'Select Department'.
- Select Course:** A dropdown menu with the placeholder 'Select Course'.
- Register:** A blue button with white text.

The browser's address bar shows 'localhost:5173', and the page title is 'Course Registration System'.

The screenshot shows a web browser window with the address bar displaying 'localhost:5173'. The page content is a form titled 'Course Registration System'. The form has the following fields and values:

- Student Name: Ajay I
- Register Number: 71812401007
- Email: ajay.2401007@sec.ac.in
- Department: Computer Science and Engineering
- Select Course: Machine Learning

Below the form is a blue 'Register' button. Below the button is a green 'Registration Successful' message. The message contains the following details:

- Name: Ajay I
- Register No: 71812401007
- Email: ajay.2401007@sec.ac.in
- Department: Computer Science and Engineering
- Selected Course: Machine Learning

RESULT:

Thus, the Course Registration System was successfully developed using React JS, demonstrating component state, input handling, form validation, event handling, and dynamic rendering of the submitted student and course details.

EX.NO.12	NODE.JS AND MONGODB
DATE: 28.09.26	

AIM:

To develop a Node.js server-side application that connects with a MongoDB database and performs Add, View, Update, and Delete (CRUD) operations on student records.

ALGORITHM:

1. Create a Node.js project using NPM.
2. Install the required packages: express and mongodb.
3. Import the required Node.js modules.
4. Establish a connection between Node.js and MongoDB.
5. Create a student collection containing register number, name, department, email, and CGPA.
6. Implement routes to add, view, update, and delete student records.
7. Start the Node.js server.
8. Test the CRUD operations using a browser or API testing tool.
9. Display the appropriate response for each operation.

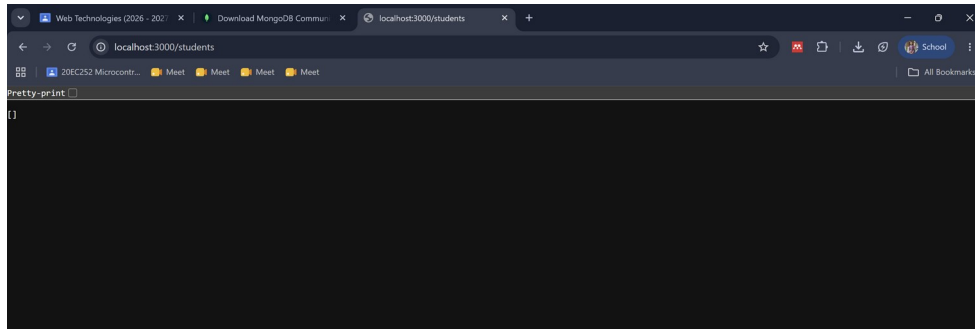
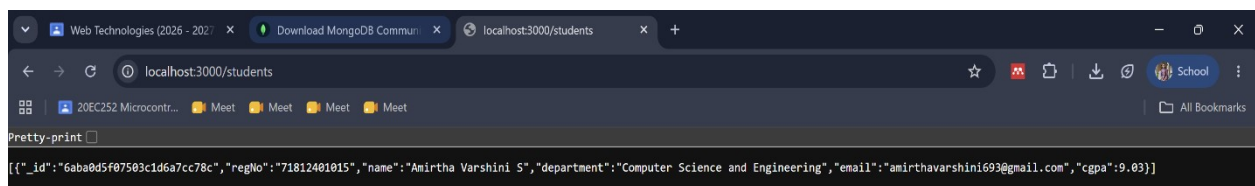
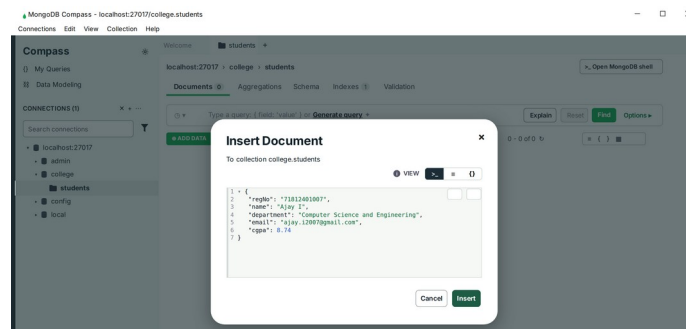
CODE:**Server.js:**

```
const express = require("express");
const { MongoClient } = require("mongodb");
const app = express();
app.use(express.json());
const client = new MongoClient("mongodb://127.0.0.1:27017");
async function start() {
  await client.connect();
  const db = client.db("college");
  const students = db.collection("students");
  app.post("/add", async (req, res) => {
    await students.insertOne(req.body);
    res.send("Student added successfully");
  });
  app.get("/students", async (req, res) => {
    res.json(await students.find().toArray());
  });
  app.put("/update/:regNo", async (req, res) => {
    await students.updateOne(
      { regNo: req.params.regNo },
      { $set: req.body }
    );
    res.send("Student updated successfully");
  });
  app.delete("/delete/:regNo", async (req, res) => {
    await students.deleteOne({ regNo: req.params.regNo });
    res.send("Student deleted successfully");
  });
  app.listen(3000, () => {
    console.log("Server running at http://localhost:3000");
  });
}
```

```
}  
start();
```

JSON:

```
{  
  "regNo": "71812401007",  
  "name": "Ajay I",  
  "department": "Computer Science and Engineering",  
  "email": "ajay.i2007@gmail.com",  
  "cgpa": 8.74  
}
```

OUTPUT:**ADD:**

VIEW:

Student CRUD

71812401007

Ajay I

CSE

ajay.2401007@srec.ac.in

8.74

Add View Update Delete

```
{
  "_id": "6aba0d5f07503c1d6a7cc78c",
  "regNo": "71812401007",
  "name": "Ajay I",
  "department": "Computer Science and Engineering",
  "email": "ajay.12007@gmail.com",
  "cgpa": 8.74
}
```

UPDATE:

Student CRUD

71812401007

Ajay I

CSE

ajay.2401007@srec.ac.in

8.74

Add View Update Delete

Student updated successfully

DELETE:

MongoDB Compass - localhost:27017/college.students

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

localhost:27017

admin

college

students

config

local

Welcome

students

localhost:27017 > college > students

Documents (1) Aggregations Schema Indexes (1) Validation

Type a query: { field: 'value' } or Generate query +

Explain Reset Find Options

ADD DATA

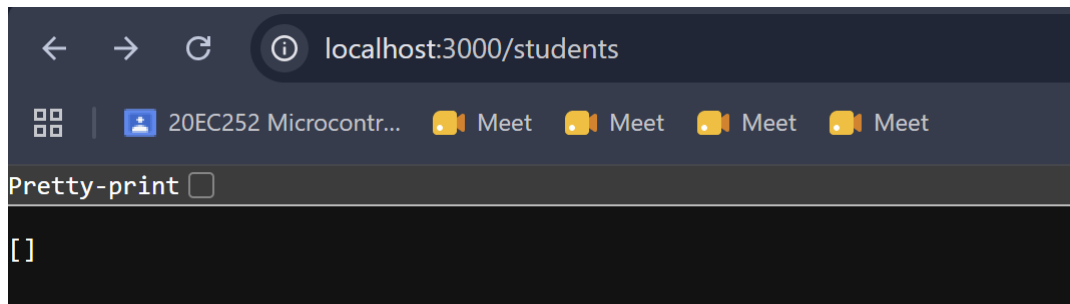
0/0 of 0

{ }

```
{
  "_id": ObjectId("6aba0d5f07503c1d6a7cc78c"),
  "regNo": "71812401007",
  "name": "Ajay I",
  "department": "Computer Science and Engineering",
  "email": "ajay.12007@gmail.com",
  "cgpa": 8.74
}
```

Document flagged for deletion.

CANCEL DELETE

**RESULT:**

Thus, a Node.js and MongoDB student record management system was successfully developed and the CRUD operations were implemented using Node.js, Express, NPM packages, and MongoDB.